#4 Since s is not the empty string, it can be broken into a *head* character, c (the first character of s) and a *tail*, the rest of the string.
#5 If there is an outgoing edge in node for the character c, we can keep recursively traversing the tree.
#6 Following the recursive definition of this method, we need to insert tail in the subtree referenced by the edge marked with character c.
#7 Otherwise, it means that we cannot traverse the trie any further: now we have to add the remaining characters in s (be careful: not just in `tail`!) as a new branch.

This last operation is implemented in a different utility method, that is implemented using (surprise!) recursion.

The definition of the method is quite straightforward: as shown in the right half of figure 6.8, we just consume one character of the remaining string, create a new edge marked with this character and a brand-new, empty node `N` at the other side of the edge, and then recursively add the tail of the string to the tree `T` rooted at `N`[14].

**Listing 6.6 method addNewBranch**

```
function addNewBranch(node,
s)                                                    #1
  if s == “” then
                                                        #2
    node.keyNode ←
true                                                            #3
    return

  c, tail ←
s.splitAt(0)
```